Ayudantía 3

Repaso para la PEP 1: NumPy, Matplotlib, pandas y clases

Física Computacional II – Ayudante: Martín Estay Valdés

Objetivos

- Simular datos con `np.random.default_rng` y analizarlos con operaciones vectorizadas, máscaras booleanas y *broadcasting*, sin ciclos sobre los elementos.
- Detectar tránsitos en una curva de luz con el criterio $F < \bar{F} - 3\sigma$ y estimar su profundidad, el cociente $R_p/R_\star$ y el periodo orbital.
- Construir imágenes astronómicas sobre una grilla de píxeles, medir distancias entre fuentes y hacer fotometría de apertura con máscaras circulares.
- Organizar un catálogo en un `DataFrame` de pandas: limpiar datos faltantes o inválidos, crear columnas, agrupar con `pd.cut` y `groupby` y guardar resultados.
- Escribir una clase con `__init__`, `@property` y `@classmethod` que reúna el análisis de un catálogo estelar.
- Aplicar la ley de Stefan–Boltzmann y la definición de magnitud para decidir qué estrellas son visibles para un instrumento.
- Producir figuras completas con `plt.subplots`: ejes con unidades, escalas logarítmicas, leyendas y barras de color.

1 Teoría

NumPy: máscaras, índices y broadcasting

Una comparación entre un arreglo y un número devuelve un arreglo booleano del mismo largo. Al indexar con él se conservan solo los elementos `True`, y `mascara.sum()` cuenta cuántos son. Las condiciones se combinan con `&`, `|` y `~`, siempre entre paréntesis.

Para ubicar extremos se usa `np.argmax`/`np.argmin`, que devuelven el *índice* y no el valor; `np.argsort` entrega los índices que ordenan el arreglo de menor a mayor (`[::-1]` invierte el orden). En una matriz, `np.argmax` entrega un índice "aplanado" que se convierte en (fila, columna) con `np.unravel_index`.

Dos arreglos de formas distintas pueden operarse si sus dimensiones son compatibles: iguales o una de ellas igual a 1. Con `None` (o `np.newaxis`) se agrega un eje de largo 1:

```
t = np.linspace(0, 10, 6) # forma (6,)
c = np.array([2.0, 7.0]) # forma (2,)
dif = t[:, None] - c[None, :] # forma (6, 2): todas las diferencias
cerca = np.any(np.abs(dif) < 1, axis=1) # forma (6,)
```

Un *slice* de un arreglo es una vista: modificarlo cambia el original. Si se quiere un arreglo independiente se usa `.copy()`.

Curvas de luz y tránsitos

Cuando un planeta pasa frente a su estrella, el flujo observado disminuye en

$$\Delta F = 1 - F_{\text{tránsito}} \simeq \left(\frac{R_p}{R_\star}\right)^2 . \tag{1}$$

Con ruido gaussiano de desviación σ, un punto con $F < \bar{F} - 3\sigma$ es poco probable que se deba solo al ruido (alrededor de 1 en 740), por lo que se toma como candidato a tránsito. Si los puntos seleccionados forman grupos separados en el tiempo, cada grupo corresponde a un tránsito; `np.diff` sobre los tiempos permite encontrar dónde termina uno y empieza el siguiente. El periodo P es la separación entre los centros de tránsitos consecutivos, y la *curva plegada* se obtiene graficando el flujo en función de la fase

$$\phi = \big[(t - t_0 + P/2) \bmod P\big] - P/2, \tag{2}$$

que en Python se escribe con el operador `%`.

Imágenes y grillas de píxeles

Una imagen es una matriz donde la fila corresponde a y y la columna a x. Las coordenadas de cada píxel se obtienen con `np.meshgrid`, `np.mgrid` (grilla completa) o `np.ogrid` (grilla abierta, de formas `(alto,1)` y `(1,ancho)`, que se combinan por broadcasting). Una estrella puede modelarse con un perfil gaussiano

$$I(x, y) = I_0 \exp\left[-\frac{(x - x_0)^2 + (y - y_0)^2}{2\sigma^2}\right]. \tag{3}$$

Si las posiciones y amplitudes de n estrellas se escriben con forma `(n,1,1)` y la grilla con forma `(1,alto,ancho)`, el resultado es un "cubo" `(n,alto,ancho)` y la imagen final es su suma sobre el eje 0.

El flujo total de una gaussiana es $2\pi I_0 \sigma^2$, y el flujo contenido dentro de un radio R es

$$F(< R) = 2\pi I_0 \sigma^2 \left[1 - \exp\left(-\frac{R^2}{2\sigma^2}\right)\right]. \tag{4}$$

La *fotometría de apertura* suma los píxeles dentro de un círculo de radio R, restando antes el fondo. Para eso se construye una máscara con $(x - x_0)^2 + (y - y_0)^2 \le R^2$. Las distancias entre todos los pares de fuentes se obtienen con `scipy.spatial.distance.pdist`, y `squareform` las ordena en una matriz simétrica.

pandas

Un `DataFrame` es una tabla con columnas con nombre. Lo esencial para la prueba:

```
df = pd.DataFrame({"id": ids, "T_K": T}) # desde un diccionario
df.head(); df.info(); df.describe() # exploración
df["T_K"]; df.loc[df["T_K"] > 7000, ["id", "T_K"]] # selección por condición
df.iloc[0] # selección por posición
df["L"] = df["R"]**2 * (df["T_K"] / 5772)**4 # nueva columna (vectorizada)
df.isna().sum(); df.dropna(); df.fillna(0) # datos faltantes
df.sort_values("L", ascending=False); df["L"].idxmax()
df["grupo"] = pd.cut(df["T_K"], bins=[0, 5000, np.inf], labels=["fría", "caliente"])
df.groupby("grupo", observed=True).agg(n=("id", "size"), T_media=("T_K", "mean"))
df.to_csv("salida.csv", index=False); pd.read_csv("salida.csv")
```

La media de una columna booleana es la fracción de valores `True`. Al filtrar un `DataFrame` que luego se va a modificar conviene usar `.copy()`.

Clases

Una clase reúne atributos y métodos. `__init__` se ejecuta al crear la instancia y `self` es la instancia misma. Un método decorado con `@property` se usa como atributo (sin paréntesis), y uno con `@classmethod` recibe la clase `cls` en lugar de `self`, por lo que sirve para construir instancias de otras formas.

```python
class Catalogo:
    """Catálogo de estrellas.

    Parameters
    ----------
    df : pandas.DataFrame
        Tabla con una fila por estrella.
    """
    def __init__(self, df):
        self.datos = df.copy()

    @classmethod
    def desde_csv(cls, ruta):
        return cls(pd.read_csv(ruta))

    @property
    def n(self):
        return len(self.datos)

cat = Catalogo.desde_csv("datos.csv")
cat.n # sin paréntesis
```

Luminosidad y magnitudes

Una estrella de radio R y temperatura efectiva T que emite como cuerpo negro tiene luminosidad $L = 4\pi R^2 \sigma_{SB} T^4$. En unidades solares,

$$\frac{L}{L_\odot} = \left(\frac{R}{R_\odot}\right)^2 \left(\frac{T}{T_\odot}\right)^4, \qquad T_\odot = 5772 \text{ K}. \tag{5}$$

La magnitud bolométrica absoluta y la aparente, con d en parsec, son

$$M_{\rm bol} = 4.74 - 2.5 \log_{10}\left(\frac{L}{L_\odot}\right), \qquad m_{\rm bol} = M_{\rm bol} + 5\log_{10} d - 5. \tag{6}$$

La escala es inversa: un objeto más brillante tiene una magnitud *menor*. Un telescopio con magnitud límite $m_{\rm lim}$ ve las estrellas con $m_{\rm bol} \le m_{\rm lim}$.

2 Ejercicios

En cada ejercicio defina el generador al comienzo, una sola vez, con la semilla indicada. Todas las figuras deben tener etiquetas en los ejes con unidades y leyenda o barra de color cuando corresponda.

Ejercicio 1

Use `rng = np.random.default_rng(7)`. Genere 1500 tiempos equiespaciados entre 0 y 30 días y un flujo normalizado igual a 1 con ruido gaussiano de desviación 0.001. Simule los tránsitos de un planeta con centros en $t = 4$, 11, 18 y 25 días: en cada uno, los puntos con $|t - t_c| < 0.2$ días disminuyen su flujo en 0.008.
Comandos: `np.linspace`, `rng.normal`, `np.arange`, `np.abs`, `np.any`, `np.mean`, `np.std`, `np.sqrt`, `np.diff`, `np.where`, `np.split`, `%`, `ax.axhline`, `ax.axvline`, `plt.subplots`.

(a) Construya la máscara de los puntos en tránsito usando broadcasting entre el arreglo de tiempos y el arreglo de centros, sin ciclos.

(b) Calcule el flujo medio, la desviación estándar y el umbral $\bar{F} - 3\sigma$. ¿Cuántos puntos quedan bajo el umbral? ¿Por qué σ resulta mayor que 0.001?

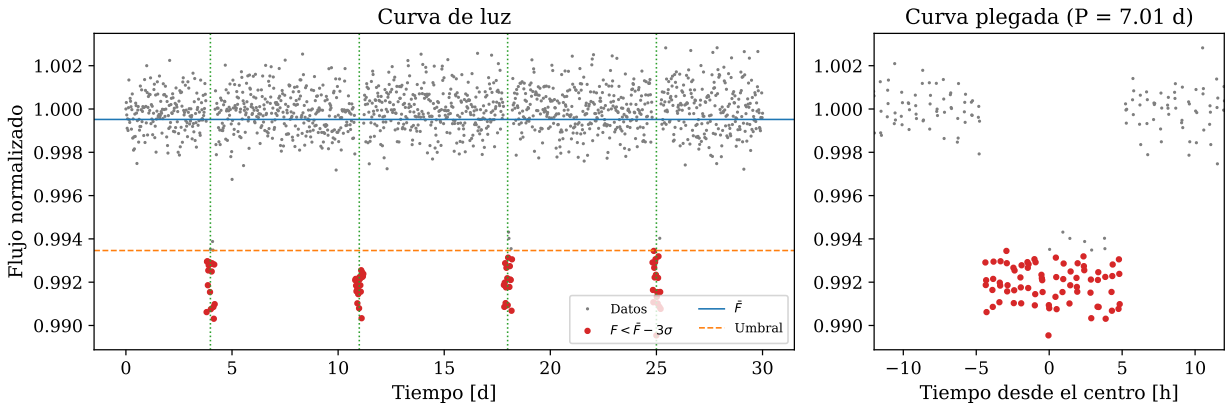
(c) Estime la profundidad del tránsito y el cociente $R_p/R_\star$.

(d) Separe los puntos detectados en grupos, cortando donde dos tiempos consecutivos están separados por más de 1 día. Descarte los grupos con menos de 3 puntos, calcule el centro de cada grupo y estime el periodo.

(e) Escriba una función `graficar_curva(t, F, mascara, umbral, centros, P)` que genere una figura de dos paneles: a la izquierda la curva de luz completa, con los puntos detectados destacados, las líneas $\bar{F}$ y $\bar{F} - 3\sigma$, y líneas verticales en los centros estimados; a la derecha la curva plegada con el periodo estimado, en horas. ¿Qué ventaja tiene la curva plegada?

Salida esperada:

```
Flujo medio      : 0.99952
Desv. estándar   : 0.00202
Umbral (3 sigma): 0.99346
Puntos bajo el umbral: 73
Profundidad      : 0.0081
Rp/R*            : 0.090
Centros [d]: [ 3.98 11.   17.99 25.  ]
Periodo estimado: 7.006 d
```



Ejercicio 2

Use `rng = np.random.default_rng(11)`. Simule una imagen de 120×120 píxeles con 5 estrellas gaussianas de $\sigma = 2.5$ px. Genere, en este orden, las posiciones x_0 e y_0 uniformes entre 15 y 105 px y las intensidades I_0 uniformes entre 50 y 200. Agregue un fondo constante de 5 y ruido gaussiano de desviación 1.
Comandos: `np.mgrid`, `np.ogrid`, `None`, `np.exp`, `.sum(axis=0)`, `np.argmax`, `np.unravel_index`, `np.argsort`, `pdist`, `squareform`, `np.fill_diagonal`, `pd.DataFrame`, `ax.imshow`, `ax.contour`, `plt.Circle`, `fig.colorbar`.

(a) Construya el cubo `(5, 120, 120)` con broadcasting y súmelo para obtener la imagen. Ubique el píxel más brillante de la imagen. ¿Coincide con la estrella de mayor I_0?

(b) Numere las estrellas del 1 al 5 de mayor a menor I_0. Calcule la matriz de distancias entre sus centros y encuentre el par más cercano.

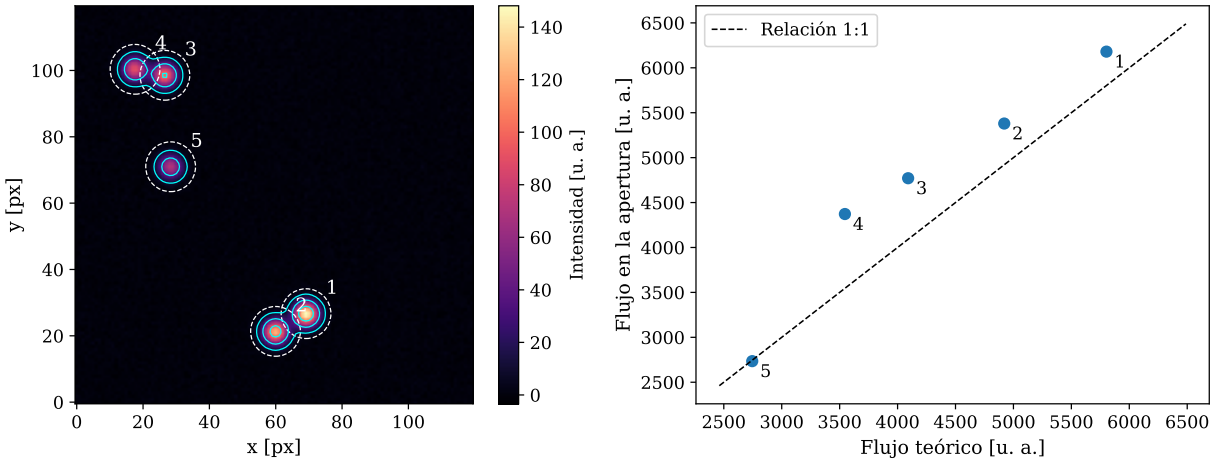
(c) Escriba una función `mascara_circular(forma, centro, radio)` con `np.ogrid`, donde `centro` sea (x, y). Mida el flujo de cada estrella en una apertura de radio $R = 3\sigma$ (restando el fondo) y guarde los resultados en un `DataFrame` con las columnas `estrella`, `x0`, `y0`, `I0`, `n_pix` y `F_ap`.

(d) Agregue la columna `F_teo` con el flujo teórico dentro de la apertura y la columna `cociente` $= F_{\mathrm{ap}}/F_{\mathrm{teo}}$. ¿Por qué algunas estrellas tienen un cociente claramente mayor que 1? Relacione su respuesta con la matriz de distancias.

(e) Haga una figura de dos paneles: la imagen sin fondo con contornos del modelo, las aperturas y el número de cada estrella; y el flujo medido en función del teórico con la recta 1:1.

Salida esperada:

```
Forma del cubo  : (5, 120, 120)
Pixel más brillante: x=69, y=27, valor=153.1
Par más cercano: estrellas 3 y 4, separación 9.2 px
 estrella     x0      y0      I0  n_pix     F_ap   F_teo  cociente
        1 69.13   26.68 149.43    176 6179.98 5802.78      1.07
        2 59.94   21.34 126.71    177 5379.60 4920.55      1.09
        3 26.57   98.54 105.35    176 4770.80 4091.09      1.17
        4 17.58  100.35  91.30    175 4372.41 3545.37      1.23
        5 28.31   70.97  70.70    178 2734.78 2745.35      1.00
```



Ejercicio 3

Use `rng = np.random.default_rng(2026)`. Construya un `DataFrame` de 300 estrellas con las columnas `id` (`S001`, ..., `S300`), `R_Rsol` $\sim \mathcal{U}(0.3, 3)$, `T_K` $\sim \mathcal{U}(3000, 12000)$ y `d_pc` $\sim \mathcal{U}(5, 200)$, generadas en ese orden. Luego reemplace por `np.nan` la temperatura de 12 estrellas elegidas con `rng.choice(300, 12, replace=False)` y cambie el signo de la distancia de otras 6 elegidas de la misma forma. Guarde la tabla en `datos/poblacion.csv`, creando la carpeta desde el código.
Comandos: `rng.choice`, `.loc`, `to_csv`, `read_csv`, `select_dtypes`, `isna`, `notna`, `.all(axis=1)`, `reset_index`, `np.log10`, `idxmax`, `pd.cut`, `groupby`, `agg`, `invert_xaxis`, `ax.hist`, `Path.mkdir`.

Implemente una clase `Poblacion`, documentada en formato NumPy Docstring, con:

(a) `__init__(self, df)`, que guarde una copia del `DataFrame` y llame a `limpiar()`; un `@classmethod` `desde_csv(ruta)`; y un `@property` `n` con el número de estrellas.

(b) `limpiar()`: informa cuántos valores faltan en cada columna numérica y elimina las filas con `NaN` o con valores no positivos. ¿Por qué se eliminan 17 filas y no 18?

(c) `calcular()`: agrega las columnas `L_Lsol`, `M_bol` y `m_bol`.

(d) `clasificar(m_lim=10.0)`: agrega la columna booleana `visible` y devuelve la fracción de estrellas visibles. Compare $m_{\mathrm{lim}} = 10$ con $m_{\mathrm{lim}} = 8$.

(e) `resumen_por_tipo()`: clasifica por temperatura en los tipos M (< 3700 K), K (< 5200), G (< 6000), F (< 7500), A (< 10000) y B, y devuelve, para cada tipo, el número de estrellas, la temperatura media, la luminosidad mediana y la fracción visible. ¿Por qué las estrellas M casi nunca son visibles?

(f) `graficar(m_lim)`: figura de dos paneles con el diagrama HR (luminosidad logarítmica, temperatura invertida, visibles y no visibles en distinto color) y el histograma de m_{bol} con una línea en m_{lim}. ¿Se observa una secuencia principal? ¿Por qué?

(g) `guardar(ruta, solo_visibles=False)`: guarda la tabla en CSV, creando la carpeta, y devuelve el número de filas guardadas.

Por último, compruebe que la clase también funciona con un `DataFrame` escrito a mano con el Sol ($d = 4.848 \times 10^{-6}$ pc), Sirio ($R = 1.71\,R_\odot$, $T = 9940$ K, $d = 2.64$ pc) y Próxima Centauri ($R = 0.154\,R_\odot$, $T = 3042$ K, $d = 1.30$ pc).

Salida esperada:

```
Valores faltantes: {'T_K': 12} (4.0 % de las filas)
Filas eliminadas: 17 de 300
Estrellas válidas: 283
Fracción visible (m_lim=10): 0.799
       n     T_media  L_mediana  frac_visible
tipo
M     20   3392.4747     0.2221        0.1500
K     53   4467.5399     0.9228        0.5660
G     22   5625.7439     3.1947        0.8636
F     44   6733.9375     4.9264        0.7955
A     78   8704.7989    12.4772        0.9359
B     66  10996.0544    51.9158        1.0000
Fracción visible (m_lim=8): 0.502
Guardadas: 226 | leídas: (226, 9)
     id  L_Lsol  M_bol   m_bol  visible
    Sol   1.000  4.740 -26.832     True
  Sirio  25.718  1.214  -1.678     True
Proxima   0.002 11.584   7.154     True
```

